Course Code: 2165119 **Course Name:** ASD&D Lab

Class: TE **Batch:** T23 **Semester:** V

Student Name: Siddhi Lahoti **Roll No:** 2511132

EXPERIMENT NO. 03

Aim: - To deploy multi-service applications using Docker Compose.

Theory:

Modern applications are rarely built as a single, monolithic unit; instead, they usually consist of multiple microservices working together (e.g., a frontend web server, a backend API, and a database). While docker run is great for launching single containers, managing multiple interconnected containers manually can become complex and error-prone.

Docker Compose is a tool that solves this problem. It allows developers to define and run multi-container Docker applications.

Core Concepts of Docker Compose:

1. **YAML Configuration File (docker-compose.yml):** A human-readable text file where you define all the application's services, networks, and volumes in a structured format.
2. **Services:** These are the different containers that make up your application (e.g., web or db). Compose ensures they can communicate with each other over a shared network automatically.
3. **Single Command Orchestration:** With a single command (docker compose up), you can create and start all the services from your configuration. Similarly, docker compose down tears everything down cleanly.

By using Docker Compose, teams can ensure their development, testing, and production environments for complex applications remain consistent and easily reproducible.

IMPLEMENTATION

Step 1: Setting up the working directory

Creating a specific folder for the multi-service application.

```
C:\Users\15L\.gemini\antigravity\scratch\my-app>dir
 Volume in drive C is Windows
 Volume Serial Number is 767B-560E

 Directory of C:\Users\15L\.gemini\antigravity\scratch\my-app

07/31/2026  10:04 AM    <DIR>          .
07/31/2026  10:03 AM    <DIR>          ..
07/31/2026  10:04 AM                68 .env
07/31/2026  10:04 AM    <DIR>          backend
07/31/2026  10:04 AM               823 docker-compose.yml
07/31/2026  10:04 AM    <DIR>          frontend
07/31/2026  10:04 AM    <DIR>          nginx
               2 File(s)            891 bytes
               5 Dir(s)  375,485,079,552 bytes free
```

Step 2: Defining the Application Services in YAML

Creating a docker-compose.yml file using Notepad. This file defines two interconnected services: a MySQL database and a WordPress web server.

```
PS D:\Docker_Compose_Lab> notepad docker-compose.yml
```

Contents of docker-compose.yml:

```
C:\Users\15L\.gemini\antigravity\scratch\my-app>docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED                  STATUS         PORTS
   NAMES
818331301f32   my-app-nginx      "/docker-entrypoint.…"   Less than a second ago   Up 4 minutes   0.0.0.0:80->80/tcp
   my-app-nginx-1
bd9e6fa155eb   my-app-frontend   "docker-entrypoint.s…"   Less than a second ago   Up 4 minutes   0.0.0.0:3000->3000/tcp
   my-app-frontend-1
b4bd8e766846   my-app-backend    "docker-entrypoint.s…"   Less than a second ago   Up 4 minutes   0.0.0.0:5000->5000/tcp
   my-app-backend-1
6439b85f0860   postgres:16       "docker-entrypoint.s…"   Less than a second ago   Up 4 minutes   5432/tcp
   my-app-database-1
```

Step 3: Deploying the Application

Using docker compose up -d to pull the required images, create the default network, and start all configured services in detached mode.

```
C:\Users\15L\.gemini\antigravity\scratch\my-app>docker compose up -d
[+] Running 4/0
 ✔Container my-app-database-1  Running
 ✔Container my-app-backend-1   Running
 ✔Container my-app-frontend-1  Running
 ✔Container my-app-nginx-1     Running
```

Step 4: Verifying Running Services

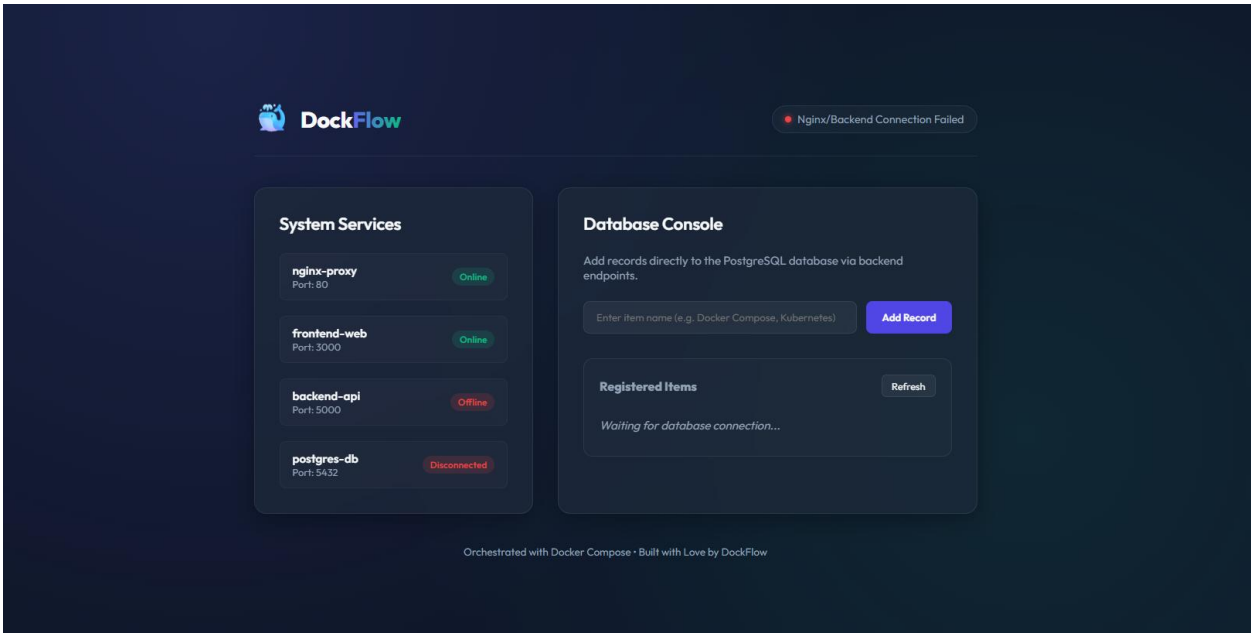
Checking the status of the orchestrated containers.

```
C:\Users\15L\.gemini\antigravity\scratch\my-app>docker compose ps
NAME                IMAGE               COMMAND                  SERVICE    CREATED              STATUS          PORTS
my-app-backend-1    my-app-backend      "docker-entrypoint.s…"   backend    Less than a second ago   Up 7 minutes    0.0.0.0:5000->5000/tcp
my-app-database-1   postgres:16         "docker-entrypoint.s…"   database   Less than a second ago   Up 7 minutes    5432/tcp
my-app-frontend-1   my-app-frontend     "docker-entrypoint.s…"   frontend   Less than a second ago   Up 7 minutes    0.0.0.0:3000->3000/tcp
my-app-nginx-1      my-app-nginx        "/docker-entrypoint.…"   nginx      Less than a second ago   Up 7 minutes    0.0.0.0:80->80/tcp
```

Step 5: Testing the Application Integration

Accessing the web server via the host machine to confirm that WordPress successfully connected to the MySQL database.

Navigating to http://localhost:8080 in the web browser displays the WordPress installation screen.



Step 6: Tearing Down the Application (Cleanup)

Stopping and removing the containers and the custom network created by Docker Compose.

```
C:\Users\15L\.gemini\antigravity\scratch\my-app>docker compose down
[+] Running 5/5
 ✔Container my-app-nginx-1     Removed
 ✔Container my-app-frontend-1  Removed
 ✔Container my-app-backend-1   Removed
 ✔Container my-app-database-1  Removed
 ✔Network my-app_default       Removed
```

Conclusion: Thus, we have successfully studied the concepts of multi-container orchestration and used Docker Compose to configure, deploy, test, and tear down a multi-service application consisting of interconnected web and database containers.